

Supervisor - Patch - Exercise

Section 1: All 58 bugs found & resolved across the system

→ Bug 1: SQL Operator precedence leaking Archived tasks (Critical)
→ Location: backend/src/main/java/internal/taskTracker/TaskRepository.java (line 14-20),

→ How found: Inspected raw SQL and tested search with seed data. Tasks #20 and #21 have archived = True. When searching for their terms matching their title/description, they leaked into results.

→ Root Cause: In SQL, AND has higher precedence than OR. The query `Where archived = false and title LIKE :term OR description LIKE :term AND status = :status` was evaluated as `(archived = FALSE AND title LIKE) OR (description LIKE AND status = :status)`. Any archived task matching description leaked, and status filtering was bypassed for title matches.

→ Fix & Why: Added explicit parentheses: `WHERE archived = FALSE AND (Lower(title) LIKE :term OR LOWER(description) LIKE :term) AND (:status IS NULL OR status = :status)`. Enforces strict boolean precedence.

→ Bug 2: Leading wildcard search was bypassing indexes

→ Where: TaskController.java

→ How I find it: checked the query execution plan.

→ Why it happened: Using %term% at the start of LIKE prevents normal B-tree index use and can force full table scans on large datasets.

→ What I Changed: Normalize the search term once and keep wildcard handling consistent. For 100K+ records, use a full-text search solution such as H2/Lucene.

→ Bug 3: Redundant toLowerCase() before SQL LOWER()

→ Where: TaskController.java.

→ How I find it: Noticed java was lowercasing the term while SQL also used LOWER().

→ Why it happened: That creates unnecessary string work and can introduce locale/collation differences.

→ What I changed: Normalize the query term once instead of repeating the same operation.

→ Bug 4: Missing indexes for filter and sort columns

→ Where: schema.sql

→ How I found it: Noticed java was lowercasing the term while SQL also

→ Why it happened: status was just VARCHAR, so invalid values such as INVALID could be stored.

→ What i changed: Added indexes for status and for archived + created_at DESC.

→ Bug 5: Missing CHECK constraints for enum values

→ Where: schema.sql

→ How I find it: Reviewed database column definitions.

→ Why it happened: status was just VARCHAR, so invalid values such as INVALID could be stored.

→ What i changed: Added ~~indexes for status~~ database CHECK constraints for valid status values (OPEN, IN_PROGRESS, DONE) and priority values.

→ Bug 6: No updated_at audit timestamp

→ Where: schema.sql and Task.java

→ How I found it: Compared the entity and database table.

→ Why it happened: Only created_at existed, so modification time could not be tracked reliably.

→ What i changed: Added updated_at and @UpdateTime
This also helps with cache invalidation.

→ Bug 7: Artificial Thread.sleep() blocked Tomcat threads.

→ where: TaskController.java

→ How I found it: Initial Loading took about 1 second even with only 47 rows. A Thread.sleep() was found in the controller.

→ Why it happened: An empty search could block one servlet thread for 1 second. With 200 Tomcat threads, enough concurrent requests could exhaust the whole pool.

→ What I changed: Removed the artificial sleep and the related complexity calculation.

→ Bug 8: IllegalArgumentException From TaskStatus.valueOf()

→ Where: TaskController.java

→ How I find it: Initial loading took about 1 second even with only 47 rows. A Thread.sleep() Calling ? status = INVALID produced a 500 response and exposed a stack trace.

→ Why it happened: valueOf() throws IllegalArgumentException for unknown enum values, and it was not handled.

→ what it changed: catch the error and return HTTP 400 with the valid status values.

→ Bug 9: In memory pagination could exhaust heap

- Where: TaskTracker.java and TaskController.java
- How I find it: Reviewed the pagination implementation.
- Why it happened: The repo loaded the complete dataset and the controller then used `subList()`. Around 100,000 rows, this can create heavy heap/GC pressure and even `OutOfMemoryError`.
- What i changed: Moved pagination into the database using Spring Data Pageable/Page.

→ Bug 10: Integer overflow in pagination offset

- Where: TaskController.java
- How I found it: Tested a boundary such as `page = Integer.MAX_VALUE`.
- Why it happened: $(page - 1) * pageSize$ could overflow to a negative integer and break indexing.
- What i changed: Clamped the maximum page numbers to 10,000.

→ Bug 11: page and pageSize were not validated

- Where: TaskController.java
- How i found it: Tested `page = -5` and `pageSize = 999999`.
- Why it happened: Bad values could break pagination logic or cause unnecessarily large allocation.

→ What i changed: clamp page to atleast 1 and pageSize between 1 and 100.

→ Bug 12: System.out.println used instead of structured logging

→ Where: TaskController.java

→ How i found it: found direct console output in the request handler.

→ Why it happened: System.out lacks log levels and makes standard log shipping / management harder.

→ What i changed: Replaced it with SLF4J parameterized logging.

→ Bug 13: JPA entity was exposed directly through the API

→ Where: TaskDto.java and TaskController.java.

→ How i found it: Controller responses were returning the raw Task entity.

→ Why it happened: This leaks internal persistence details and couples the API contract to the database model.

→ What i changed: Created TaskDto and map entities to DTOs before returning them.

→ Bug 14: Missing service layer

→ Where: TaskService.java and Task Controller.java

→ How i found it: The Controller was directly calling repository methods.

→ Why it happened: Business logic and persistence were too tightly coupled, weakening separation of responsibility.

→ What i changed: Added a TaskService with @Service for business operations and transactions.

- Bug 15: Status stored as a plain String
- Where: Task.java
 - How I found it: Inspected the status field.
 - Why it happened: A string allows invalid values and does not use JPA's enum mapping.
 - What I changed: changed it to `@Enumerated(EnumType.STRING)` with TaskStatus.
- Bug 16: Entity had no equals()/hashCode()
- Where: Task.java
 - How I found it: Audited the JPA entity contract.
 - Why it happened: default object identity can cause duplicate-detection problems in HashSet/HashMap.
 - What I changed: Implemented equals() and hashCode() using the persistent id.
- Bug 17: Missing validation constraints on entity fields.
- Where: Task.java
 - How I found it: checked the entity annotations.
 - Why it happened: Titles could be null and descriptions were not properly bounded before persistence.
 - What I changed: Added non-null constraints, sensible length limits and `@PrePersist` defaults.
- Bug 18: Read-only API Surface was intentionally kept.
- Where: TaskController.java
 - How I found it: Reviewed the endpoint design against the assignment scope.
 - Why it happened: The template was meant for querying not for creating or editing tasks.

→ What i changed: Kept the API read-focused. This avoids scope creep and follows the instruction to patch, not rebuild.

→ Bug 19: Missing @Transactional(readOnly = true) optimization

→ Where: TaskRepository.java and TaskService.java

→ How i found it: Profiled database read operations.

→ Why it happened: Normal Hibernate transactions can still do dirty-checking work even when nothing is changed.

→ What i changed: Use readOnly transactions for read paths to avoid unnecessary flush/snapshot work.

→ Bug 20: H2 web console was enabled without authentication

→ Where: application.properties.

→ How i found it: found spring.h2.console.enabled=true in the default configuration.

→ Why it happened: An exposed unauthenticated SQL console creates a serious security/RCE risk.

→ What i changed: Disable the H2 console by default and isolated it to development settings.

→ Bug 21: CORS origin was hardcoded.

→ Where: TaskController.java and application.properties.

→ How i found it: Inspected the @CrossOrigin configuration.

→ Why it happened: Hardcoded localhost:5173 works only locally and breaks in staging/production.

→ What i changed: Made the allowed origin configurable through app.cors.allowed-origins.

> Bug 22: CRLF log injection through the search query

- where: TaskController.java.
- How i found it: Audited values directly into logs.
- Why it happened: A malicious query containing CR/LF characters could forge extra log entries in SIEM/log systems.
- what i changed: Sanitized carriage-return and newline characters before logging.